# Project Initialization and Planning Phase

| | |
|---|---|
| Date | 28 June 2026 |
| Team ID | 6a3a555ae5895b2c9759029d |
| Project Title | EduGenie - Google Gemini Powered Learning Assistant<br>Prediction for Loan Approval |
| Maximum Marks | 5 Marks |

## Project Proposal (Proposed Solution) report

The proposal report aims to transform loan approval using machine learning, boosting efficiency and accuracy. It tackles system inefficiencies, promising better operations, reduced risks, and happier customers. Key features include a machine learning-based credit model and real-time decision-making.

| | |
|---|---|
| **Project Overview** | |
| Objective | Give students a single AI-powered assistant that answers questions, explains concepts at the right depth, generates practice quizzes, summarizes notes, and builds a personalized learning roadmap - all backed by Google Gemini with an offline/local-model fallback.<br>A FastAPI web application with five request/response modules (chat, explanation, quiz, summary, learning path), a Jinja2/HTML/CSS/JS frontend, JSON-file persistence for query/response history, and a Dockerized deployment path. |
| Scope | process, incorporating machine learning for a more robust and<br>Students lack a single, adaptive tool that can explain a concept at their level, quiz them on it, summarize their notes, and tell them what to study next - they currently stitch this together from multiple generic tools. |
| **Problem Statem** | |
| Description | Solving this reduces study friction and time-to-understanding, gives students a low-cost self-testing mechanism, and helps them plan their learning path with less guesswork.<br>customer satisfaction. |
| Impact | Use Google Gemini (with dynamic best-model resolution and retry/backoff) as the primary generative engine, prompted with strict JSON response schemas per feature, and a local HuggingFace LaMini-Flan-T5 model plus a built-in mock/demo mode as fallbacks so the app is always usable. |
| **Proposed Soluti** | |
| Approach | Employing machine learning techniques to analyze and predict<br>CPU / GPU for running the local T5 model (CUDA auto-detected, uses GPU if available).<br>RAM to hold the local LaMini-Flan-T5 model in memory when loaded (2-4 GB recommended). |
| Key Features | Disk space for model weight cache, Docker image, and JSON database files (2-3 GB). |

FastAPI, Uvicorn, Jinja2 template rendering engines.

google-generativeai, transformers, torch, pydantic, python-dotenv, httpx, python-multipart, pytest.

| | |
|---|---|
| | - Real-time decision-making for quicker loan approvals.<br>- Continuous learning to adapt to evolving financial landscapes. |

## Resource Requirements

| Resource Type | Description | Specification/Allocation |
|---|---|---|
| **Hardware** | | |
| Computing Resources | CPU/GPU specifications, number of cores | CPU / GPU for running local T5 (CUDA auto-detected) |
| Memory | RAM specifications | 2-4 GB RAM recommended for local model loading |
| Storage | Disk space for data, models, and logs | 2-3 GB for model weights cache and data logs |
| **Software** | | |
| Frameworks | Python frameworks | FastAPI application server with Uvicorn |
| Libraries | Additional libraries | google-generativeai SDK, transformers, PyTorch, pydantic |
| Development Environment | IDE | Visual Studio Code, python-dotenv, Git/GitHub |
| **Data** | | |
| Data | Source, size, format | Local structured JSON flat-file storage logs |